UNIVERSITÉ DE MOUNDOU —-TCHAD

Master en Informatique — Data Science et Intelligence Artificielle

RAPPORT DE TP D'EXAMEN — PARTIE 1

Gestion d'une ligne de commande

Application web (Laravel)

Réalisé par : Mahamat Issa et Ibrahim Adam Fadoul

Laravel 12 · PHP 8.3 · MySQL · Bootstrap · Chart.js

12 octobre 2026

Sommaire

1. Introduction et objectifs
2. Analyse du sujet et base de données
3. Architecture et choix technologiques
4. Réalisation des cinq travaux
5. Validation et scénarios de test
6. Difficultés rencontrées et solutions
7. Limites et perspectives
8. Conclusion

Annexe — Extraits de code

1. Introduction et objectifs

Ce rapport présente la partie web du TP d'examen « Gestion d'une ligne de commande ». Le sujet impose une base de données relationnelle (clients, commandes, produits, régions et départements) et demande cinq travaux : un formulaire d'enregistrement des produits d'une commande, l'affichage des listes, la modification d'un produit, la suppression d'un produit dans une commande, et un tableau de bord statistique accompagné d'une analyse de performance.

L'application a été développée avec le framework Laravel et une base MySQL. Elle contient toute la logique métier : les règles de gestion du stock, les transactions et le calcul des statistiques. Les objectifs étaient de :

- traduire fidèlement le schéma relationnel de l'énoncé dans MySQL, avec toutes les contraintes d'intégrité ;
- réaliser les quatre premiers travaux avec des règles de gestion fiables (cohérence du stock, opérations « tout ou rien ») ;
- concevoir un tableau de bord utile à la décision et justifier ses choix par une analyse de performance chiffrée.

Une application mobile Flutter, qui réutilise cette application web par l'intermédiaire d'une API, fait l'objet d'un second rapport.

2. Analyse du sujet et base de données

2.1 Travaux demandés et réalisation

N°	Travail demandé	Réalisation web
1	Formulaire d'enregistrement des produits d'une commande donnée	Page /commandes/create : choix du client et de la date, lignes produit/quantité ajoutées dynamiquement.
2	Afficher la liste des produits, des commandes et des clients	Pages /produits, /commandes et /clients, paginées.
3	Modifier un produit	Page /produits/{id}/edit, accessible par le bouton « Modifier ».
4	Supprimer un produit dans une commande	Bouton « Supprimer » sur le détail d'une commande (/commandes/{id}).
5	Tableau de bord sur la ligne de commande et analyse de performance	Page /dashboard : indicateurs, trois graphiques, alertes de stock, tableau d'analyse de performance.

2.2 Schéma relationnel

La base de données imposée par l'énoncé comporte six relations (# désigne une clé étrangère) :

Relation	Attributs	Clé primaire
Departement	idDep, nomDep	idDep
Region	idRegion, nomRegion, #idDep	idRegion
Client	idCli, nomCli, prenomCli, email, photo, #idRegion	idCli
Produit	idProd, designProd, qteStock, prixUProd, photoProd	idProd
Commande	idCmd, dateCmd, #idCli	idCmd
ligneCommande	#idCmd, #idProd, qteCmd	(idCmd, idProd) — clé composée

Les cardinalités sont : un département contient plusieurs régions (1,N) ; une région regroupe plusieurs clients (1,N) ; un client passe plusieurs commandes (1,N) ; une commande contient plusieurs produits et un produit apparaît dans plusieurs commandes (N,N), cette association étant matérialisée par la table ligneCommande qui porte la quantité commandée.

2.3 Implémentation dans MySQL

- Les six tables sont créées par une migration Laravel, dans l'ordre des dépendances (départements, régions, clients, produits, commandes, lignes de commande).
- Les clés primaires gardent les noms de l'énoncé (idCli, idCmd, idProd, idRegion, idDep) ; chaque modèle Eloquent déclare sa clé via la propriété $primaryKey.
- La table ligne_commandes a une clé primaire composée (idCmd, idProd) : un produit ne peut figurer qu'une seule fois dans une commande.
- Des contraintes de clé étrangère garantissent l'intégrité : la suppression d'une commande supprime ses lignes (cascade), et un produit déjà commandé ne peut pas être supprimé.
- Des index sont posés sur les clés étrangères et sur dateCmd, pour accélérer les jointures et les regroupements par période.

- Le prix unitaire est stocké en DECIMAL(12,2) plutôt qu'en nombre flottant, afin d'éviter les erreurs d'arrondi sur les montants en FCFA.
- Le moteur InnoDB est imposé dans la configuration : il est indispensable pour les clés étrangères et les transactions utilisées par le formulaire de commande.
- Un jeu de données de test (seeder) génère au départ 5 départements, 6 régions, 30 clients, 12 produits et 80 commandes réparties sur environ dix mois.

2.4 Base obtenue (vérification sous phpMyAdmin)

La base tp_commande a été contrôlée dans phpMyAdmin. Elle contient 16 tables : les 6 tables métier de l'énoncé (clients, commandes, departements, ligne_commandes, produits, regions) et 10 tables techniques créées par Laravel (cache, cache_locks, failed_jobs, jobs, job_batches, migrations, password_reset_tokens, personal_access_tokens, sessions, users). Toutes sont en InnoDB. La ligne de total affiche MyISAM, qui est le moteur par défaut du serveur : c'est ce qui a rendu nécessaire d'imposer InnoDB (voir section 6).

Après les essais, la base contient 30 clients, 5 départements, 6 régions, 14 produits, 81 commandes et 199 lignes de commande : deux produits et une commande ont été ajoutés pendant les tests, en plus du jeu de données de départ.

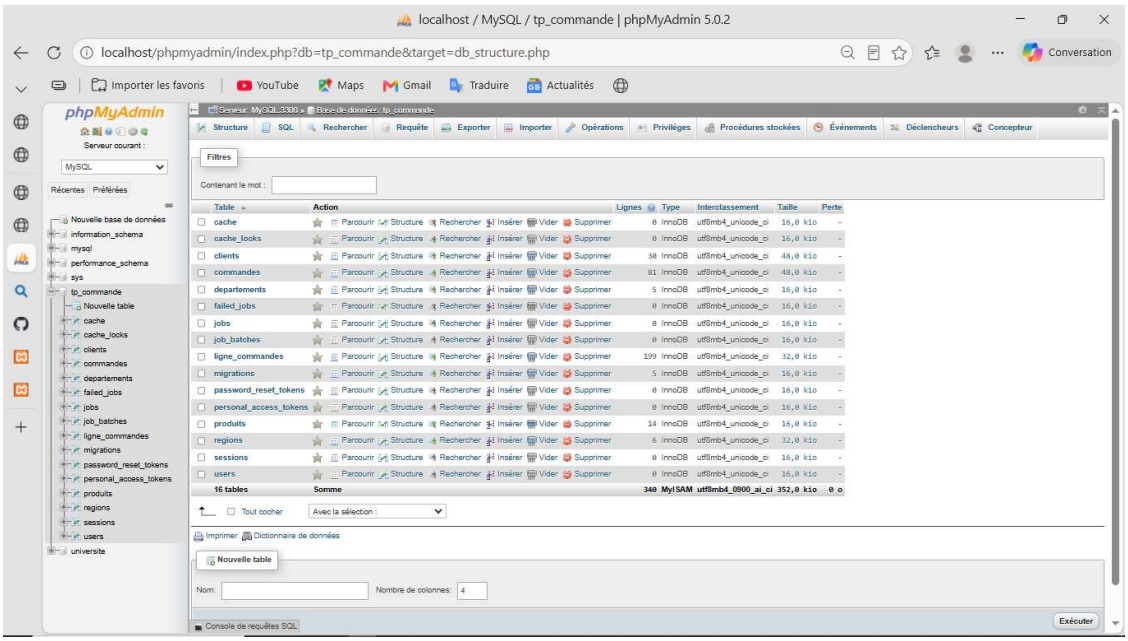


Figure 1 — phpMyAdmin — structure de la base tp_commande (16 tables InnoDB)

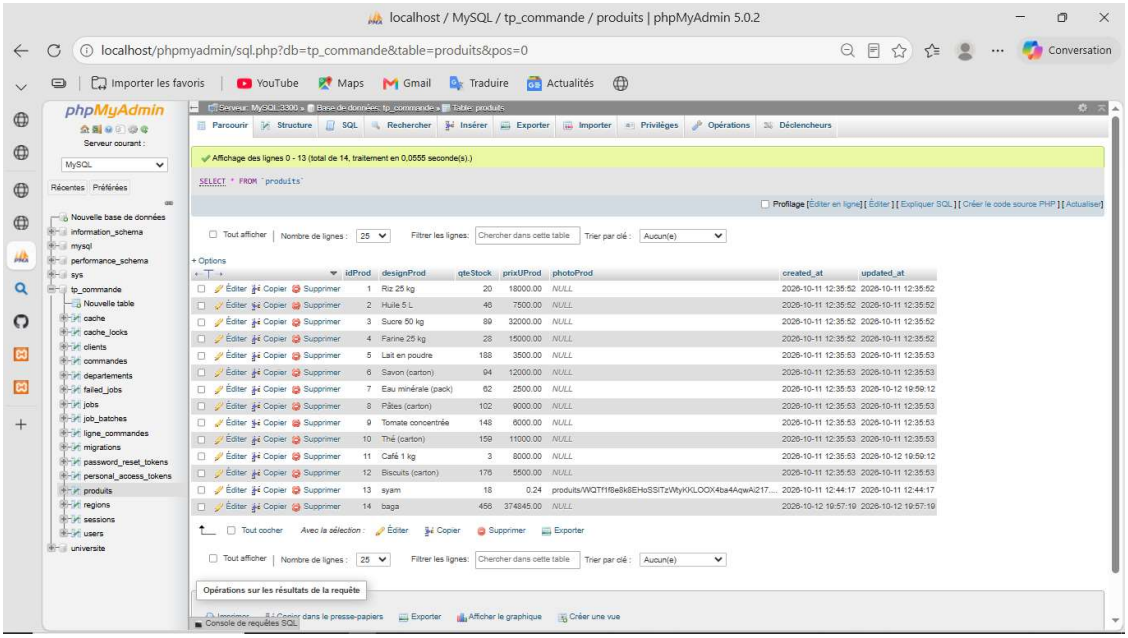


Figure 2 — phpMyAdmin — contenu de la table produits après les essais

3. Architecture et choix technologiques

L'application suit l'architecture MVC de Laravel : les routes dirigent chaque requête vers un contrôleur, qui utilise les modèles Eloquent pour interroger MySQL, puis renvoie une vue Blade (page HTML) au navigateur.

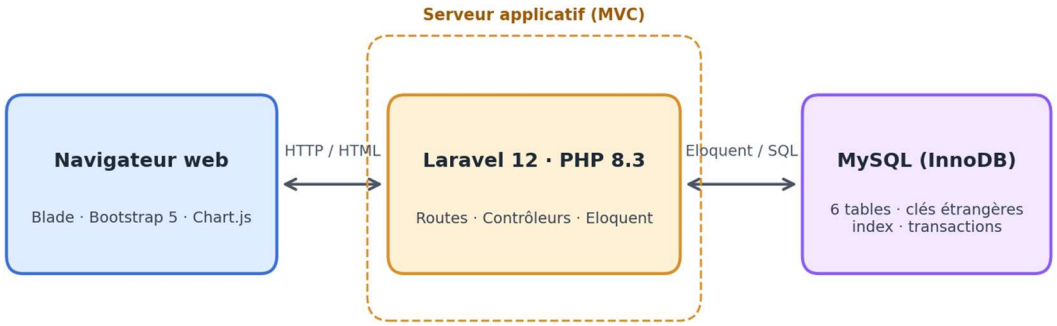


Figure 3 — Architecture de l'application web

Couche	Technologie	Justification
Backend	Laravel 12, PHP 8.3	Structure MVC, ORM Eloquent, migrations, validation, transactions.
Base de données	MySQL (InnoDB)	Base relationnelle imposée ; clés étrangères et transactions.
Vues	Blade, Bootstrap 5	Pages responsives, formulaires et tableaux homogènes.
Graphiques	Chart.js	Graphiques interactifs légers, chargés depuis un CDN.

Organisation du code

Emplacement	Contenu
database/migrations	Création des 6 tables (clés, index, contraintes).
database/seeders/TpSeeder.php	Jeu de données de test.
app/Models	Client, Commande, Produit, Region, Departement (relations Eloquent).
app/Http/Controllers	ProduitController, CommandeController, ClientController, DashboardController.
resources/views	Vues Blade : mise en page, produits, commandes, clients, tableau de bord.
routes/web.php	Routes de l'interface web.

4. Réalisation des cinq travaux

4.1 Modèles et relations

- **Commande** : appartient à un Client ; possède plusieurs Produit via la table pivot ligne_commandes (attribut qteCmd) ; un accesseur calcule le total de la commande.
- **Produit** : relié aux commandes par la même table pivot.
- **Client** : appartient à une Region et possède plusieurs commandes ; **Region** appartient à un Departement.

4.2 Routes et contrôleurs

URL	Méthode	Action	Travail
/commandes/create, /commandes	GET, POST	CommandeController@create, store	1
/produits, /commandes, /clients	GET	index de chaque contrôleur (pagination)	2
/produits/{produit}/edit, /produits/{produit}	GET, PUT	ProduitController@edit, update	3
/commandes/{commande}/produits/{produit}	DELETE	CommandeController@retirerProduit	4
/dashboard	GET	DashboardController@index	5

4.3 Travail 1 — Formulaire d'enregistrement d'une commande

Le formulaire permet de choisir un client et une date, puis d'ajouter dynamiquement autant de lignes (produit + quantité) que nécessaire, grâce à un petit script JavaScript. À l'enregistrement, le contrôleur applique les règles de gestion suivantes :

- validation des données : client existant, date valide, au moins une ligne, produits existants et distincts, quantité entière supérieure ou égale à 1 ;
- exécution dans une transaction (« tout ou rien ») : si une ligne échoue, rien n'est enregistré ;
- verrouillage de la ligne produit (lockForUpdate) pour éviter que deux commandes simultanées ne dépassent le stock ;
- contrôle du stock : si la quantité demandée dépasse le stock, la commande est refusée avec un message explicite ;
- en cas de succès, ajout de la ligne dans la commande et décrémentation du stock du produit.

L'extrait de code correspondant est donné en annexe (A.1).

4.4 Travail 2 — Listes des produits, des commandes et des clients

Les trois listes sont paginées et chargent leurs relations en une seule fois (chargement anticipé) afin d'éviter des requêtes inutiles. La liste des produits affiche le stock et le prix unitaire en FCFA, avec un bouton « Modifier » ; la liste des commandes affiche le numéro, la date, le client, le nombre de produits et le total, avec un bouton « Détail » ; la liste des clients affiche l'identité, l'email, la région et le département.

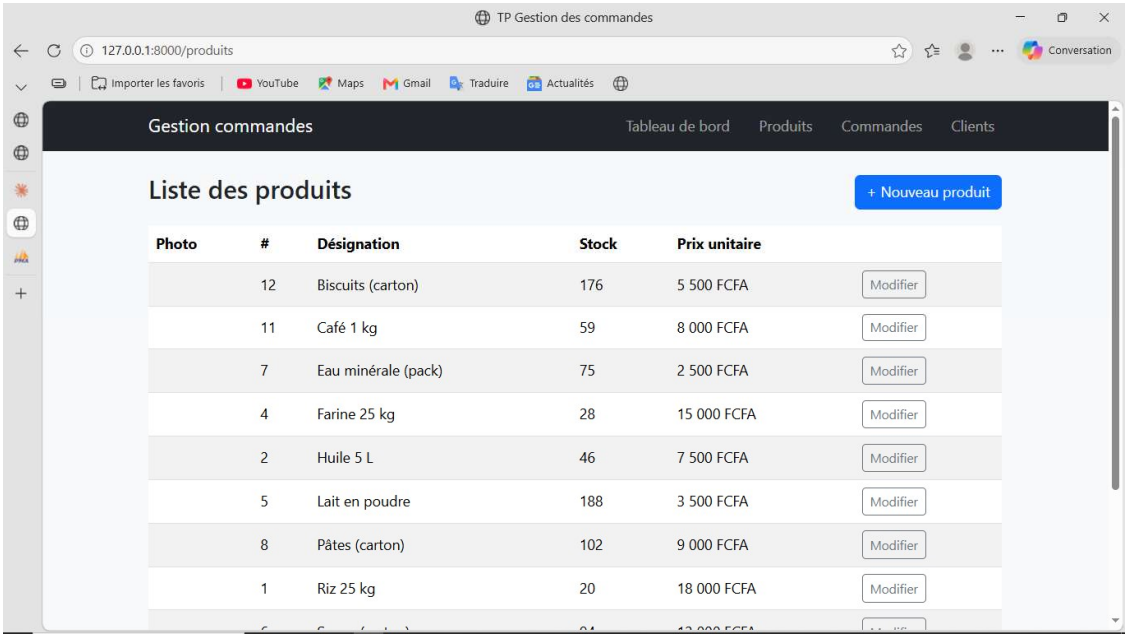


Figure 4 — Web — liste des produits

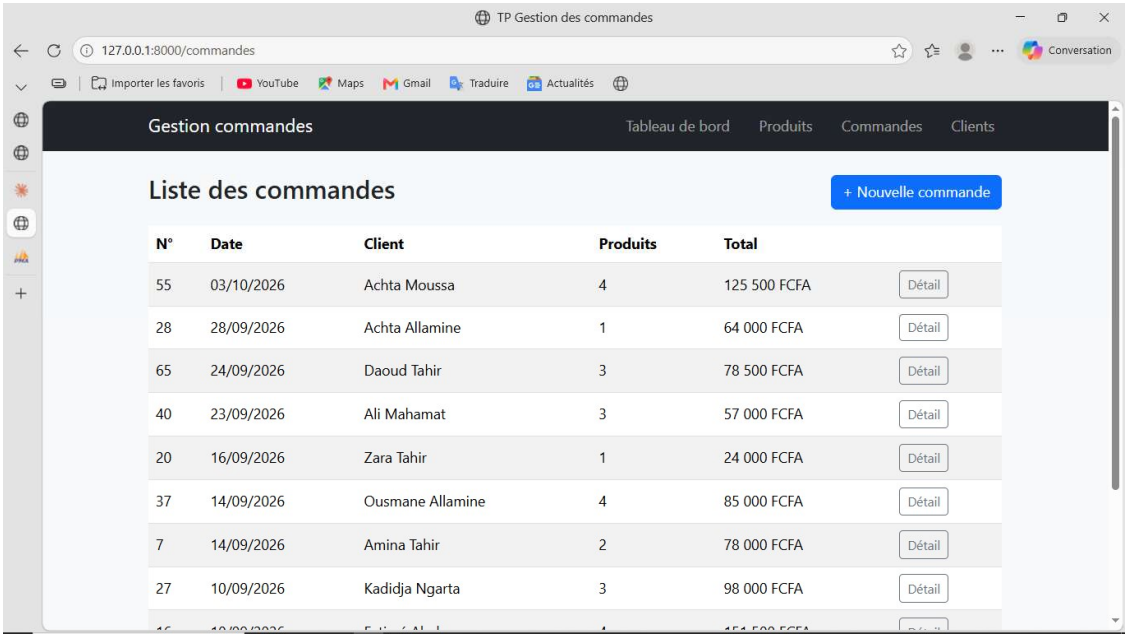


Figure 5 — Web — liste des commandes

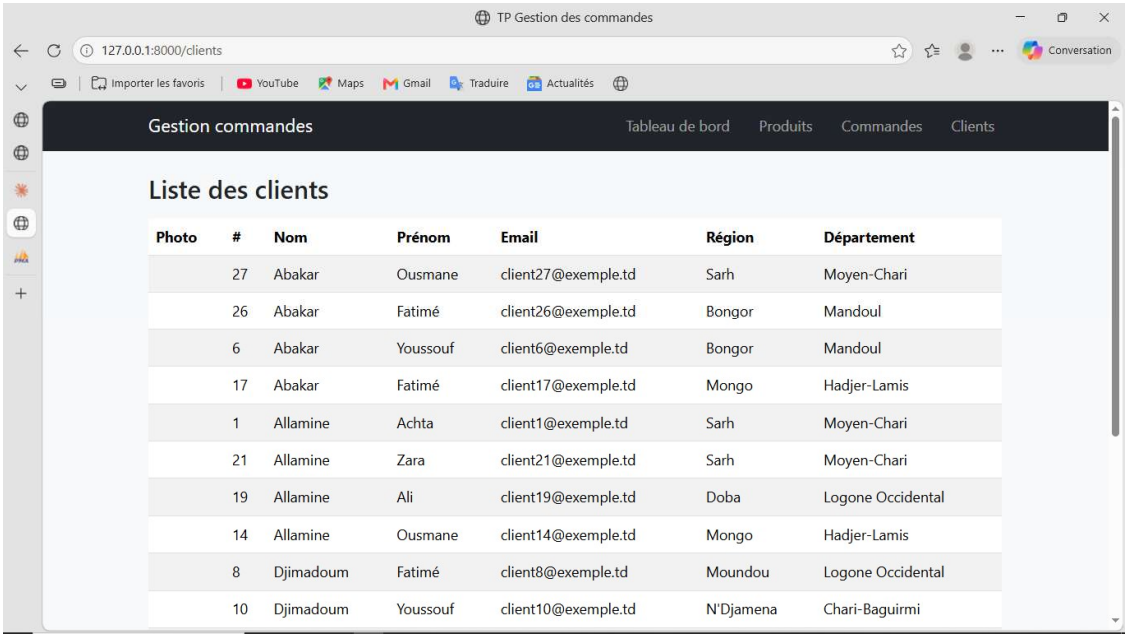


Figure 6 — Web — liste des clients

4.5 Travail 3 — Modification d'un produit

Le bouton « Modifier » ouvre un formulaire pré-rempli (désignation, quantité en stock, prix unitaire, photo). Les champs sont validés côté serveur (désignation obligatoire, stock entier positif, prix numérique positif, image de moins de 2 Mo) et les erreurs sont réaffichées dans la page. Si une nouvelle photo est envoyée, l'ancienne est supprimée du disque. La création d'un produit utilise le même formulaire.

4.6 Travail 4 — Suppression d'un produit dans une commande

Sur le détail d'une commande, chaque ligne possède un bouton « Supprimer » avec demande de confirmation. L'opération s'exécute dans une transaction : la quantité de la ligne est d'abord restituée au stock du produit, puis la ligne est retirée de la commande (détachement de la relation). Le stock reste ainsi cohérent avec les commandes existantes (extrait en annexe A.2).

4.7 Travail 5 — Tableau de bord

Le tableau de bord est volontairement orienté « décision » : il répond aux questions que se pose un gestionnaire sur ses lignes de commande. Il est composé de :

- quatre indicateurs clés : chiffre d'affaires, nombre de commandes, panier moyen et unités vendues ;
- un graphique en barres du top 5 des produits par chiffre d'affaires ;
- un graphique par région (répartition du chiffre d'affaires) ;
- un graphique de l'évolution mensuelle du chiffre d'affaires ;
- une alerte de stock faible (produits dont le stock est inférieur à 10) ;
- un tableau d'analyse de performance (section 4.8).

Toutes les statistiques sont calculées directement par MySQL avec des agrégats (SUM de qteCmd × prixUProd, GROUP BY) plutôt que par des boucles PHP, ce qui est plus rapide et plus économe en mémoire (extrait en annexe A.3).

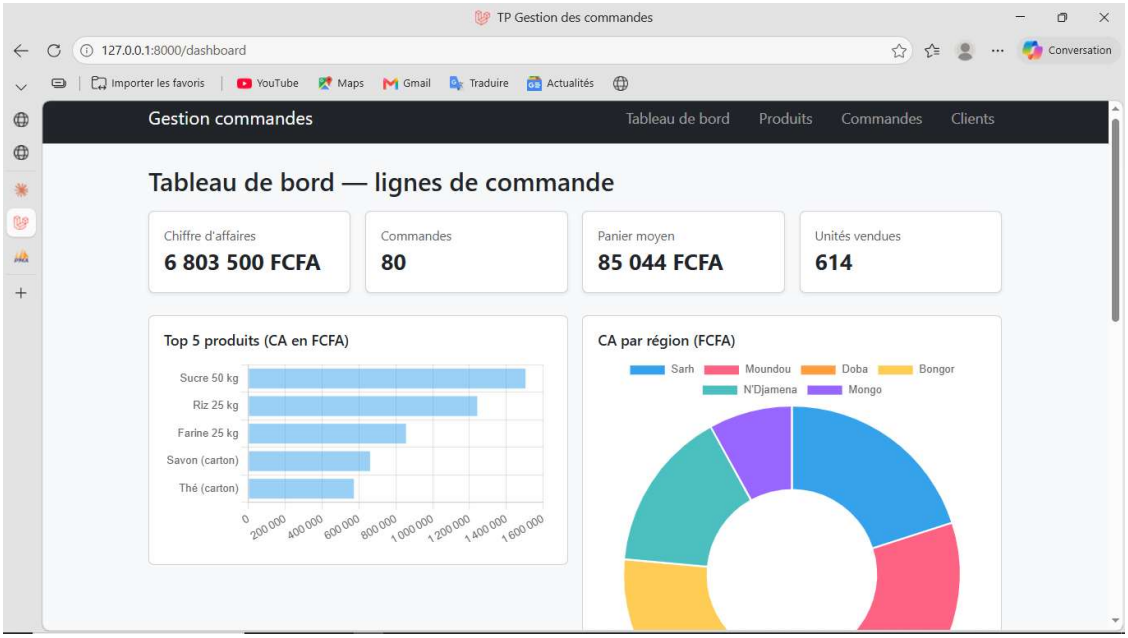


Figure 7 — Web — tableau de bord : indicateurs, top 5 produits et CA par région

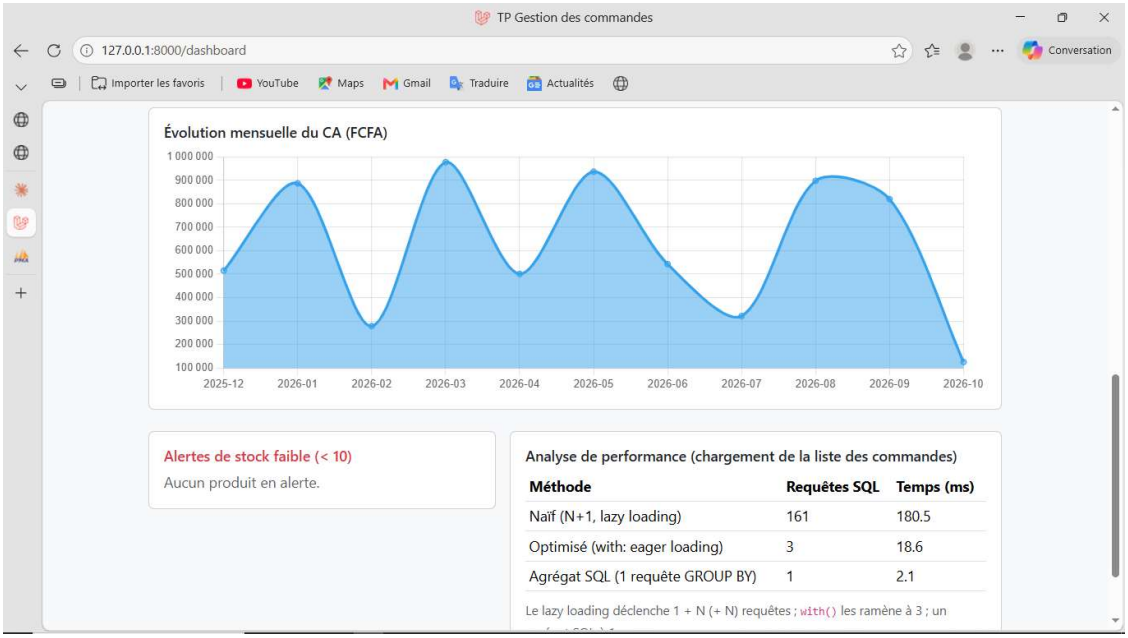


Figure 8 — Web — tableau de bord : évolution mensuelle, alertes de stock et analyse de performance

Lecture des résultats

Les valeurs suivantes correspondent à l'état initial des données (80 commandes), avant les essais décrits en section 5.

Indicateur	Valeur
Chiffre d'affaires total	6 803 500 FCFA
Nombre de commandes	80
Panier moyen	85 044 FCFA
Unités vendues	614

- **Produits :** le sucre 50 kg (1 504 000 FCFA), le riz 25 kg (1 242 000 FCFA) et la farine 25 kg (855 000 FCFA) génèrent à eux trois environ 53 % du chiffre d'affaires. Les ventes sont donc concentrées sur quelques produits de base.

- **Régions :** Sarh (1 364 000 FCFA, soit 20 %), Moundou, Doba et Bongor sont proches les unes des autres ; Mongo ferme la marche avec 545 000 FCFA (8 %), ce qui indique un potentiel commercial à développer dans cette région.
- **Évolution mensuelle :** le chiffre d'affaires est irrégulier, avec des pics en janvier, mars (977 500 FCFA), mai et août, et des creux en février (278 500 FCFA) et juillet. Le dernier mois (octobre 2026) est incomplet.
- **Stock :** aucun produit n'était sous le seuil d'alerte. Le riz 25 kg, deuxième produit le plus vendu, n'avait cependant que 20 unités en stock : c'est le premier candidat à un réapprovisionnement.

4.8 Analyse de performance

L'énoncé suggère une analyse de performance. Elle porte ici sur un cas concret : le chargement de la liste des 80 commandes avec leur client et leurs produits. Trois méthodes sont comparées, et le tableau de bord mesure lui-même le nombre de requêtes SQL (journal de requêtes de Laravel) et le temps d'exécution :

Méthode	Requêtes SQL	Temps (ms)	Gain
Naïve (chargement à la demande, problème N+1)	161	180,5	—
Optimisée (chargement anticipé avec with())	3	18,6	environ ×10
Agrégat SQL (une seule requête GROUP BY)	1	2,1	environ ×86

Interprétation. Dans la méthode naïve, Laravel exécute 1 requête pour les commandes, puis 1 requête par commande pour son client et 1 requête par commande pour ses produits, soit 1 + 80 + 80 = 161 requêtes. Ce nombre croît linéairement avec les données : avec 10 000 commandes, on dépasserait 20 000 requêtes. Le chargement anticipé ramène le total à 3 requêtes (commandes, clients, produits). Enfin, quand on n'a besoin que de totaux, un agrégat SQL laisse la base faire le calcul en une seule requête, ce qui est la solution la plus rapide. Les temps ont été mesurés sur un poste local avec 80 commandes ; ils sont indicatifs, mais le rapport entre les méthodes est représentatif. La comparaison de code est donnée en annexe A.4.

Optimisations retenues dans l'application : chargement anticipé (with) dans toutes les listes ; agrégats SQL pour les statistiques ; index sur les clés étrangères et sur la date de commande ; pagination des listes ; sélection des seules données utiles.

5. Validation et scénarios de test

Le tableau suivant récapitule les scénarios de validation et leurs preuves. Les premiers sont établis par les captures d'écran du rapport ; les derniers correspondent aux règles de gestion implémentées dans le code.

Scénario	Comportement observé ou attendu	Preuve / emplacement
Affichage des listes avec le jeu de données de test	Produits, commandes et clients sont affichés avec leurs relations (région, département, total).	Figures 4 à 6
Tableau de bord	Indicateurs, trois graphiques, alertes de stock et analyse de performance s'affichent à partir des données réelles.	Figures 7 et 8
Enregistrement d'une commande	Le nombre de commandes passe de 80 à 81 et, au même instant (19:59:12), les stocks du Café 1 kg (59 → 3) et de l'Eau minérale (75 → 62) diminuent : c'est le comportement attendu pour une commande contenant ces deux produits.	Figures 4, 1 et 2
Création d'un produit	Deux produits ont été ajoutés (« syam », avec une photo enregistrée, et « baga ») avec horodatage de création.	Figure 2
Intégrité de la base	16 tables InnoDB ; 6 tables métier avec leurs clés et contraintes.	Figure 1
Alerte de stock faible	Un produit dont le stock passe sous 10 (ici le Café 1 kg, 3 unités) doit apparaître dans l'alerte du tableau de bord.	DashboardController (seuil de 10)
Quantité supérieure au stock	Refus avec le message « Stock insuffisant » ; aucune ligne enregistrée (transaction annulée).	CommandeController@store
Même produit saisi deux fois	Refus : un produit ne figure qu'une fois par commande (règle distinct et clé composée).	Validation et clé primaire composée
Modification d'un produit	Les nouvelles valeurs sont enregistrées après validation.	ProduitController@update
Suppression d'un produit dans une commande	La ligne disparaît et la quantité retourne au stock.	CommandeController@retirerProduit

6. Difficultés rencontrées et solutions

Difficulté	Cause	Solution
Syntaxe PHP refusée ; installation de Laravel impossible	PHP 8.0 installé, trop ancien pour Laravel 12 (Composer installait un Laravel 9 bloqué).	Installation de PHP 8.3 (Laragon) et mise en tête du PATH.
Erreur « could not find driver »	Extension pdo_mysql non activée : le fichier php.ini n'existait pas.	Création de php.ini à partir du modèle et activation des extensions (pdo_mysql, mbstring, openssl, fileinfo…).
Erreur « La clé est trop longue » à la migration	Moteur MyISAM par défaut sur le serveur MySQL (limite de 1 000 octets par clé).	Moteur InnoDB imposé dans config/database.php, puis migrate:fresh.
Erreur « La table users existe déjà »	Migration précédente interrompue à mi-chemin, laissant des tables sans trace dans la table migrations.	Commande migrate:fresh qui supprime et recrée toutes les tables.
Tables absentes au démarrage (erreur 1146)	Les migrations n'avaient pas été exécutées avant le seeder.	Exécution de php artisan migrate avant db:seed.

7. Limites et perspectives

- **Sécurité :** l'application n'a pas d'authentification ni de gestion des rôles ; une mise en production nécessiterait une connexion des utilisateurs, des droits par profil et HTTPS.
- **Statistiques :** ajout de filtres par période, région ou client, d'exports PDF/Excel et d'une mise en cache des indicateurs.
- **Suppression de produits :** une désactivation (archivage) plutôt qu'une suppression serait plus adaptée aux produits déjà commandés.
- **Qualité :** tests automatisés (PHPUnit) pour les règles de stock et de transaction, et déploiement sur un serveur.
- **Portabilité :** le regroupement par mois utilise la fonction DATE_FORMAT propre à MySQL ; il faudrait l'adapter pour un autre SGBD.

8. Conclusion

Cette application web couvre les cinq travaux demandés : formulaire d'enregistrement d'une commande, listes, modification d'un produit, suppression d'un produit dans une commande, et tableau de bord statistique avec analyse de performance. La base de données respecte le schéma de l'énoncé, avec clés composées, contraintes d'intégrité et index.

Les choix structurants sont l'usage de transactions et de verrous pour garder le stock cohérent, les agrégats SQL pour les statistiques et le chargement anticipé pour les listes. L'analyse de performance en montre l'intérêt : 161 requêtes et 180 ms pour la méthode naïve, contre 3 requêtes et 18,6 ms avec with(), et 1 requête et 2,1 ms avec un agrégat SQL.

Les pistes d'évolution portent surtout sur la sécurité, les tests automatisés et l'enrichissement des statistiques.

Annexe — Extraits de code

A.1 Enregistrement d'une commande (CommandeController@store)

```
$commande = DB::transaction(function () use ($data) {
    $commande = Commande::create(['idCli' => $data['idCli'], 'dateCmd' => $data['dateCmd']]);

    foreach ($data['lignes'] as $ligne) {
        $produit = Produit::lockForUpdate()->findOrFail($ligne['idProd']);

        if ($produit->qteStock < $ligne['qteCmd']) {
            throw ValidationException::withMessages([
                'lignes' => "Stock insuffisant pour {$produit->designProd}",
            ]);   // annule toute la transaction
        }

        $commande->produits()->attach($produit->idProd, ['qteCmd' => $ligne['qteCmd']]);
        $produit->decrement('qteStock', $ligne['qteCmd']);
    }
    return $commande;
});
```

A.2 Suppression d'un produit dans une commande (CommandeController@retirerProduit)

```
DB::transaction(function () use ($commande, $produit) {
    $ligne = $commande->produits()->where('produits.idProd', $produit->idProd)->firstOrFail();
    $produit->increment('qteStock', $ligne->pivot->qteCmd);   // restitution du stock
    $commande->produits()->detach($produit->idProd);          // suppression de la ligne
});
```

A.3 Requête d'agrégat du tableau de bord (top 5 produits)

```
// Top 5 produits par chiffre d'affaires
DB::table('ligne_commandes as l')
    ->join('produits as p', 'p.idProd', '=', 'l.idProd')
    ->join('commandes as c', 'c.idCmd', '=', 'l.idCmd')
    ->selectRaw('p.designProd as label, SUM(l.qteCmd * p.prixUProd) as ca')
    ->groupBy('p.idProd', 'p.designProd')
    ->orderByDesc('ca')->limit(5)->get();
```

A.4 Mesure de performance : méthode naïve et méthode optimisée

```
// Naïf : 1 requête pour les commandes + 1 par client + 1 par liste de produits
foreach (Commande::all() as $c) { $c->client->nomCli; foreach ($c->produits as $p) { /* ... */ }
}

// Optimisé : chargement anticipé (3 requêtes au total)
foreach (Commande::with(['client', 'produits'])->get() as $c) { /* ... */ }
```